



Part 3: Project Management

& Team Collaboration

Lidox Platform — Assignment 1 — April 2026

Team Members		
Fill in your team members below:		
Name	Ownership Area	Role
Brandon Jason Adebayo	Architecture, Documentation, Specifications	Architecture Lead
Mohamed Maher Abdalla	Frontend (Editor, AI UX, Presence)	Frontend & UX Lead
Abror Shopulatov	AI Orchestration, Prompt Engineering	AI & Orchestration Lead
Abdelmonaim Bounite	Backend API, Infrastructure, CI/CD, DevOps	Backend & DevOps Lead

1 Team Structure & Ownership

1.1 Code Ownership Areas

Each team member owns a primary area of the codebase. Ownership means: first point of contact for bugs, responsible for code review approvals in that area, and accountable for test coverage.

Ownership Area	Scope (Directories)	Key Responsibilities
Brandon — Architecture Lead	docs/, diagrams/, cross-cutting specs	System design, ADRs, C4 diagrams, requirements engineering, Mermaid diagrams, specification documents. Reviews all cross-cutting PRs.
Mohamed — Frontend & UX Lead	apps/web/	TipTap editor integration, AI suggestion overlay, presence rendering, offline IndexedDB buffer, state management, accessibility (WCAG 2.1 AA), responsive UI.
Abror — AI & Orchestration Lead	apps/api/src/ai/, apps/sync-server/	AI Orchestration Service, prompt templates, LLM provider adapter, token metering, Hocuspocus/Yjs sync server, CRDT operation relay, presence fan-out.
Abdelmonaim Bounite — Backend & DevOps Lead	apps/api/ (core), packages/types/, infra	NestJS modules (auth, documents, permissions), REST API contracts, Zod schemas, role enforcement, Docker Compose, CI/CD (GitHub Actions), Redis/PostgreSQL setup.

1.2 Cross-Cutting Feature Handling

Features that span multiple ownership areas (e.g., the AI assistant, which touches frontend, backend API, AI service, and real-time layer) are handled as follows:

- Feature lead:** The team member whose area is most affected takes the lead. For the AI assistant, Abror leads because the AI Orchestration Service is the most complex component.
- Interface contracts first:** Before implementation begins, all involved owners agree on the API contract (request/response schemas in packages/types). This contract is merged as the first PR for the feature.
- Parallel implementation:** Each owner implements their side against the agreed contract. Integration tests validate the contract.
- Integration PR:** The feature lead opens a final PR that wires everything together and runs E2E tests.

1.3 Technical Disagreement Resolution

When team members disagree on a technical choice:

- The two parties each write a brief (max 200-word) argument in the GitHub issue.
- If unresolved in 24 hours, the team holds a 15-minute timeboxed discussion.
- If still unresolved, the person whose ownership area is most affected makes the final call and documents the decision as a lightweight ADR in docs/decisions/.

2 Development Workflow

2.1 Branching Strategy

We use **GitHub Flow** (trunk-based with feature branches):

- **Main branch (main):** Always deployable. Protected: requires passing CI and at least 1 approved review.
- **Feature branches:** Named `feat/{issue-number}-short-description` (e.g., `feat/42-ai-proposal-overlay`).
- **Fix branches:** Named `fix/{issue-number}-short-description`.
- **Merge policy:** Squash merge to main. The squash commit message follows the Angular Conventional Commit format: `type(scope): description` (e.g., `feat(ai): add task-aware token budgeting`).
- **No long-lived branches:** Feature branches should be merged within 3 days. If a feature takes longer, break it into smaller PRs behind a feature flag.

2.2 Code Review Process

- **Every PR requires at least 1 approval** from a team member who is *not* the author.
- **Cross-area PRs** (touching >1 ownership area) require approval from each affected area owner.
- **Review criteria:** (1) Does it match the API contract in types? (2) Are there tests? (3) No secrets in code. (4) No `console.log` in production code. (5) TypeScript strict mode passes.
- **Review SLA:** Reviews should be completed within 24 hours. If not, the author pings in the team channel.
- **CI gates:** Linting (ESLint), type checking (TypeScript), unit tests (Jest), integration tests (Supertest) must all pass before merge.

2.3 Issue Tracking & Task Assignment

- **Tool:** GitHub Issues with project board (Kanban: Backlog → In Progress → In Review → Done).
- **Task granularity:** Each issue represents 1–3 days of work. Larger features are broken into sub-issues.
- **Assignment:** Issues are self-assigned during weekly planning. The ownership areas guide who picks what, but anyone can pick cross-cutting tasks.
- **Labels:** frontend, backend, ai, infra, cross-cutting, bug, blocked.

2.4 Communication & Decision Documentation

- **Daily:** Async standup in a dedicated Discord/Slack channel (what I did, what I'm doing, blockers).
- **Weekly:** 30-minute synchronous planning meeting to review the board and assign the next sprint's issues.
- **Decisions:** Any architectural or technical decision is documented as an ADR in `docs/decisions/` or as a comment on the relevant GitHub issue. Chat messages are ephemeral; decisions must live in the repo.

3 Development Methodology

3.1 Methodology: Scrum-Inspired Sprints (Modified)

We use a **lightweight Scrum framework** adapted for a 4-person team:

- **Sprint length:** 1 week. Short sprints force frequent integration and early detection of issues.
- **Sprint planning:** Monday, 30 minutes. Review backlog, assign issues for the week.
- **Sprint review:** Friday, 15 minutes. Demo what was completed; update stakeholders.
- **No daily standup meetings:** Replaced by async text standups (see above). A 4-person team doesn't need synchronous daily ceremonies.

3.2 Backlog Prioritization

The backlog is prioritized using a **MoSCoW** framework aligned with architectural drivers:

1. **Must Have (Sprint 1–2):** Core editing + real-time sync + basic auth + document CRUD. These are the architectural skeleton; nothing else works without them.

2. **Must Have (Sprint 3–4):** AI invocation pipeline + proposal UX + role enforcement. The product differentiator.
3. **Should Have (Sprint 5–6):** Version history, export, sharing UI, presence polish, accessibility.
4. **Could Have (Sprint 7+):** Admin dashboard, org-level settings, advanced AI features (restructure, full-document analysis).

3.3 Infrastructure & Non-Feature Work

Non-feature work (database setup, CI pipeline, testing infrastructure, Docker Compose) is explicitly scheduled in Sprint 1 as “infrastructure stories” with the same acceptance criteria rigor as feature stories. Example: “CI pipeline runs linting, type checking, and unit tests on every PR push, verified by a passing green check on a test PR.” This prevents the common antipattern of perpetually deferred infrastructure.

4 Risk Assessment

Assessment Criteria

Each risk is specific to the LidoX project. For each: description, likelihood (Low/Medium/High), impact (what breaks), mitigation strategy, and contingency plan if mitigation fails.

Risk 1: CRDT Implementation Complexity Causes Integration Delays

Likelihood: Medium. *Category:* Technical.

Impact: The core real-time editing experience is blocked. Without CRDT sync, the product has no value proposition. All downstream features (AI proposals, presence, versioning) depend on a working CRDT layer.

Mitigation: Use the mature Yjs library + Hocuspocus server rather than building a custom CRDT implementation. Start with a minimal TipTap + Yjs integration in Sprint 3 (Mar 23–26) as the first real-time deliverable.

Contingency: If Yjs integration proves too complex, fall back to a simpler OT-based approach using a central server (sacrificing offline editing initially) and revisit CRDTs in a later iteration.

Risk 2: LLM API Cost Overruns During Development and Production

AI-SPECIFIC

Likelihood: High. *Category:* AI Integration / Financial.

Impact: Development API costs spiral during testing. In production, a single viral document with heavy AI usage could generate hundreds of dollars in charges before detection.

Mitigation: (1) Use recorded fixtures (saved LLM responses) for all CI testing—zero real API calls in automated tests. (2) Implement per-org budget with hard-stop in Sprint 4 (Mar 27–30) *before* public deployment. (3) Set a \$50/month development API budget cap with team-wide alerts.

Contingency: If costs still overrun: reduce default token ceilings, remove premium-model tier (route all tasks to fast model), or temporarily disable AI features and focus on the collaboration core.

Risk 3: WebSocket Scaling Under Load

Likelihood: Medium. *Category:* Infrastructure.

Impact: As concurrent connections grow beyond a single server's capacity (~10K connections per node), users experience connection drops, missed CRDT operations, or presence lag.

Mitigation: Design the Real-Time Service as stateless from the start: CRDT state lives in Redis, not in server memory. Multiple Real-Time Service pods can serve the same document because they all relay through Redis Pub/Sub. Load testing with k6 WebSocket plugin validates capacity before each milestone.

Contingency: If Redis Pub/Sub becomes a bottleneck: shard Pub/Sub channels by document ID range, or migrate to a dedicated message broker (NATS/Kafka) for high-throughput fan-out.

Risk 4: AI Proposal Race Condition Causes Content Corruption

Likelihood: Medium. *Category:* Technical / AI Integration.

Impact: When collaborators edit text during the 2–8 s AI round-trip, proposals anchored to stale character offsets silently corrupt document content. User trust in AI features collapses.

Mitigation: The anchor-based proposal resolution system (FR-2.2, ADR-06) detects stale proposals via CRDT state vector comparison and SHA-256 hash matching. This is implemented as a core part of Sprint 4 (AI integration, Mar 27–30), not deferred as an optimization.

Contingency: If state-vector-based anchoring proves too complex: fall back to a simpler “lock the paragraph during AI processing” approach (worse UX but prevents corruption) and iterate toward the anchor-based system in a later sprint.

Risk 5: Team Coordination Breakdown on Cross-Cutting Features

TEAM-SPECIFIC

Likelihood: Medium. *Category:* Team Coordination.*Impact:* The AI assistant touches frontend (proposal overlay), backend (routing), AI service (orchestration), and real-time (WebSocket delivery). If team members implement their parts with incompatible assumptions, integration fails at the end of the sprint—a classic integration hell scenario.*Mitigation:* (1) Shared type definitions in packages/types serve as executable API contracts—if the types don't compile, the integration is broken. (2) Interface contracts are merged as the *first* PR for any cross-cutting feature, before parallel implementation begins. (3) Integration tests run in CI against all services simultaneously.*Contingency:* If integration still breaks: dedicate a full “integration sprint” where all team members work together in a single call/room to wire the feature end-to-end. Reduce parallel work in favor of sequential, paired programming until the integration pattern is established.**Risk 6: LLM Provider Outage or Rate Limiting During Demo**

AI-SPECIFIC

Likelihood: Low–Medium. *Category:* External Dependency.*Impact:* If the LLM API is down or rate-limited during a demo or grading session, the AI features appear non-functional.*Mitigation:* Implement graceful degradation (NFR-07): the editing platform remains fully functional; AI toolbar shows “temporarily unavailable.” Additionally, maintain a set of pre-recorded AI responses that can be replayed as a “demo mode” fallback.*Contingency:* Switch to a local model (e.g., Ollama with a small open-source model) for demo purposes. Quality will be lower, but the architectural pipeline (invoke→queue→process→proposal) is demonstrated end-to-end.

5 Timeline and Milestones

Milestone Acceptance Criteria

Each milestone has concrete, verifiable acceptance criteria. “Finish backend” is not a milestone; “API serves document CRUD with authentication, verified by integration tests” is.

Sprint	Dates	Deliverables	Acceptance Criteria
Sprint 1	Mar 17–19	M1: Architectural Skeleton. Monorepo setup (Turborepo). Docker Compose (PG + Redis). CI pipeline (lint + type check + unit tests). Basic NestJS API scaffolding. React + TipTap editor renders locally.	(1) turbo dev starts all 3 apps. (2) CI passes on a test PR. (3) TipTap editor renders and accepts keystrokes locally.
Sprint 2	Mar 20–22	M2: Core CRUD + Auth. Document CRUD API (POST/GET/PATCH/DELETE /api/documents). JWT auth (login, register, refresh). Frontend connects to API: create document, load document.	(1) Integration tests pass for all document endpoints. (2) Unauthenticated requests return 401. (3) Frontend creates and loads a document via the API.
Sprint 3	Mar 23–26	M3: Real-Time Sync. Hocuspocus server with Yjs. WebSocket connection from frontend. Two browser tabs sync edits in real-time. Presence (cursors) displayed.	(1) Two tabs editing the same document see each other's changes within 300 ms. (2) Cursor positions are visible across tabs. (3) Disconnecting and reconnecting restores state.
Sprint 4	Mar 27–30	M4: AI Integration. AI invocation endpoint. Job queue (BullMQ). LLM provider adapter (one provider). Proposal UX (diff overlay, accept/reject). Token metering.	(1) User selects text and invokes "Rewrite"; AI proposal appears as a diff. (2) Accept commits to CRDT; reject dismisses. (3) Token count is logged per request.
Sprint 5	Mar 31–Apr 2	M5: Permissions + Sharing. Role-based access control (Owner/Editor/Commenter/Viewer). Share document by email. Permission enforcement on API and WebSocket.	(1) Viewer cannot edit. Commenter can comment but not edit. (2) Sharing by email sends invitation. (3) Permission revocation terminates active WebSocket within 60 s.
Sprint 6	Apr 3–5	M6: Polish + Hardening. Version history UI. Export (PDF/DOCX). Offline editing. Presence collapse (>10 users). Accessibility pass. Load testing.	(1) Version history shows ≥ 5 snapshots for an actively edited document. (2) Export produces valid DOCX. (3) k6 load test: 50 concurrent editors, p95 ≤ 150 ms.

Dependency Chain

M1 (skeleton) → M2 (CRUD+Auth) → M3 (real-time sync) → M4 (AI integration) → M5 (permissions) → M6 (polish). Each milestone builds on the previous. The AI integration (M4) deliberately comes *after* real-time sync (M3) because the AI proposal conflict resolution depends on a working CRDT layer.